



Especificación de Requisitos de Software (SRS)

Rubén Martínez Pollán

Pablo Díez Fernández

Adrián García Da Silva

Contenido

1 Introducción	5
1.1 Propósito	6
1.2 Alcance	6
1.3 Personal involucrado	7
1.4 Definiciones, acrónimos y abreviaturas	7
1.5 Resumen	7
2 Descripción general	7
2.1 Perspectiva del producto	7
2.2 Funcionalidad del producto	8
2.3 Características de los usuarios	8
2.4 Restricciones	8
2.5 Suposiciones y dependencias	8
2.6 Evolución previsible del sistema	8
3 Requisitos específicos	9
3.1 Requisitos comunes de los interfaces	12
3.1.1 Interfaces de usuario	12
3.1.2 Interfaces de hardware	13
3.1.3 Interfaces de software	13
3.1.4 Interfaces de comunicación	13
3.2 Requisitos funcionales	14
3.2.1 Requisito funcional 1	15
3.2.2 Requisito funcional 2	15
3.2.3 Requisito funcional 3	15
3.2.4 Requisito funcional 4	15
3.3 Requisitos no funcionales	15
3.3.1 Requisitos de rendimiento	15
3.3.2 Seguridad	16
3.3.3 Fiabilidad	16
3.3.4 Disponibilidad	16
3.3.5 Mantenibilidad	16
3.3.6 Portabilidad	16
3.4 Otros requisitos	17
4 Apéndices	17

1 Introducción

El propósito de este documento es especificar los requisitos del sistema de planificación dinámica y proactiva "Aiyama". El sistema busca proporcionar una herramienta de agenda inteligente que, a diferencia de las tradicionales, actúe como un asistente de optimización de tiempo. Permitirá gestionar horarios, calcular huecos óptimos basados en la energía del usuario y consensuar decisiones mediante un chatbot con IA.

1.1 Propósito

El propósito de este documento es definir y especificar de forma clara, unívoca y detallada los requisitos del software a desarrollar. Servirá como contrato técnico y guía principal a lo largo de todo el ciclo de vida del proyecto, asegurando que todas las partes involucradas compartan la misma visión sobre lo que el sistema debe hacer.

Este documento está dirigido principalmente a:

- **El Cliente / Evaluadores:** Para validar que las funcionalidades descritas coinciden con las expectativas del proyecto.
- **El Equipo de Desarrollo:** Para guiar la implementación del backend (Go), el frontend (React) y la base de datos (PostgreSQL/Supabase).
- **Equipo de Pruebas (QA):** Como base para diseñar los casos de prueba y asegurar la calidad del software.

1.2 Alcance

El sistema a desarrollar es una aplicación web enfocada en la gestión eficiente y dinámica del tiempo del usuario, yendo un poco más allá que los calendarios tradicionales que ya existen. Su objetivo principal es automatizar y optimizar la distribución de tareas diarias maximizando la productividad y respetando los horarios naturales del usuario.

El sistema permitirá específicamente:

- **Gestión Integral de Tareas:** Crear, leer, actualizar y eliminar tanto bloques de tiempo fijos como tareas flexibles.
- **Personalización Biométrica:** Recopilar información clave del usuario, como su objetivo de horas de sueño y su cronotipo (mañana, tarde o noche), para adaptar las recomendaciones.
- **Organización Asistida por IA:** Interactuar con un chatbot inteligente que, apoyado en un motor de cálculo algorítmico, distribuirá de la manera más óptima las tareas flexibles teniendo en cuenta la energía requerida para cada una y la disponibilidad del calendario.
- **Sistema de Restauración de tareas:** Ofrecer una red de seguridad al usuario, permitiéndole generar y restaurar "snapshots" (fotos del estado anterior) del calendario por si no está conforme con la nueva distribución generada por el asistente. Pudiendo volver solo a la última distribución que ha tenido el usuario y no a ninguna de las anteriores a dicha distribución.

1.3 Personal involucrado

Los principales participantes en este proyecto son:

- **Usuarios finales:** Personas que utilizarán la agenda para optimizar su tiempo.
- **Equipo de desarrollo:** Responsables del análisis, diseño de la arquitectura (Go + Gemini), implementación, pruebas y despliegue de sistema.

1.4 Definiciones, acrónimos y abreviaturas

- **SRS:** Especificación de Requisitos de Software.
- **RF / RNF:** Requisito Funcional / Requisito No Funcional.
- **Gemini:** Plataforma empleada para ejecutar modelos de lenguaje de inteligencia artificial (LLM) de forma local, garantizando la privacidad de los datos del usuario.
- **Go (Golang):** Lenguaje de programación utilizado para el desarrollo del backend, la API REST y el motor de cálculo algorítmico.
- **Cronotipo:** Patrón natural de energía y vigilia de un usuario a lo largo del día (clasificado en mañana, tarde o noche), utilizado por la IA para asignar las tareas de mayor exigencia en los momentos óptimos.
- **Rollback / Snapshot:** Acción de revertir el estado del calendario a una "foto" (*snapshot*) de la distribución inmediatamente anterior, cancelando los últimos cambios generados por el asistente.
- **Bloque Fijo:** Rutina inamovible en el calendario del usuario (horario laboral, clases universitarias...) que la IA debe esquivar.
- **Tarea Flexible:** Actividad que el usuario desea realizar durante la semana, cuya ubicación exacta en el calendario es calculada y propuesta por el sistema.
- **CRUD:** Acrónimo de *Create, Read, Update, Delete* (Crear, Leer, Actualizar y Borrar). Representa las cuatro operaciones básicas de gestión de datos en la aplicación.

1.5 Resumen

Este documento se divide en varias secciones: la introducción al contexto de Alyama, la descripción general del sistema (arquitectura y restricciones), los requisitos específicos detallados (funcionales y no funcionales estructurados por módulos), y finalmente los apéndices con el modelo de datos y plan de riesgos.

2 Descripción general

2.1 Perspectiva del producto

Alyama funcionará como una aplicación interactiva que combina un backend de alto rendimiento con inteligencia artificial local. El núcleo (motor de reglas) estará programado en Go para asegurar cálculos rápidos y eficientes al cruzar variables de tiempo y energía. Se va a usar Gemini API para poder tener una api de IA online gratuita.

2.2 Funcionalidad del producto

- Onboarding: Cuestionario inicial para definir horarios fijos y niveles de energía.
- Motor de Asignación: Algoritmo en Go que cruza disponibilidad real con el perfil energético para colocar tareas flexibles.
- Flujo de Consenso: Chatbot que formula propuestas humanas (ej. "Te propongo ir al gimnasio de 20:00 a 21:00") y espera confirmación (Si/No).
- Gestión de Imprevistos: Recálculo dinámico de la semana a través de comandos de lenguaje natural.
- Control de Estado: Guardado de seguridad previo a cada cambio para permitir la función "Deshacer" (Rollback).

2.3 Características de los usuarios

El sistema está diseñado para usuarios que busquen optimizar su tiempo pero que no desean la fricción de planificar manualmente cada minuto de su día. No se requieren conocimientos técnicos avanzados; la interacción principal será conversacional, lo que hace que la curva de aprendizaje sea prácticamente nula.

2.4 Restricciones

El sistema requiere la ejecución de un modelo LLM (Gemini), lo que exigirá ciertos recursos de hardware (Memoria RAM y capacidad de procesamiento) en la máquina donde se despliegue.

Las integraciones externas (como APIs meteorológicas) quedan restringidas a fases posteriores del desarrollo por cuestiones de viabilidad temporal.

El sistema operará exclusivamente a través de navegador web; no se desarrollará aplicación móvil nativa en la versión inicial.

El idioma inicial de la interfaz y del chatbot será el español.

2.5 Suposiciones y dependencias

Se asume que el usuario proporcionará información veraz sobre sus horarios fijos durante el Onboarding.

El sistema depende de que el servicio de Gemini esté activo y correctamente configurado en el entorno de ejecución.

Se asume disponibilidad de una conexión de red estable para la comunicación entre el frontend y el backend, y entre el backend y la base de datos.

2.6 Evolución previsible del sistema

Integración con APIs meteorológicas para adaptar la planificación de tareas al aire libre según el tiempo previsto.

Soporte multiidioma: inglés y otros idiomas europeos.

Aplicación móvil nativa (iOS / Android) con notificaciones push.

Integración con calendarios externos (Google Calendar, Outlook) para sincronización bidireccional.

Motor de aprendizaje adaptativo que mejore las propuestas con base en el historial de aceptación/rechazo del usuario.

3 Requisitos específicos

Número de requisito	RF01.1
Nombre de requisito	Interfaz de inicio de sesión
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	Cliente
Prioridad del requisito	☒ Alta/ Esencial ☐ Media/ Deseado ☐ Baja/ Opcional
Descripción	El sistema debe proveer una interfaz de inicio de sesión que solicite al usuario su usuario y su contraseña.

Número de requisito	RF01.2
Nombre de requisito	Registro y configuración inicial del usuario
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	Cliente
Prioridad del requisito	☒ Alta/ Esencial ☐ Media/ Deseado ☐ Baja/ Opcional
Descripción	Bajo los campos de usuario y contraseña, la interfaz de inicio de sesión debe mostrar una opción (botón o enlace) con el texto "Registrarse", que redirija al usuario al proceso de creación de cuenta y configuración, detallado en el RF01.4.

Número de requisito	RF01.3
Nombre de requisito	Validación y Error de Autenticación
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	Cliente
Prioridad del requisito	☒ Alta/ Esencial ☐ Media/ Deseado ☐ Baja/ Opcional
Descripción	El sistema debe validar las credenciales. Si el usuario no está registrado en la base de datos o la contraseña es incorrecta, se deberá mostrar un mensaje de error claro (ej. "Usuario o contraseña incorrectos") impidiendo el acceso.

Número de requisito	RF01.4
Nombre de requisito	Creación de perfil de actividad
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	Cliente
Prioridad del requisito	☒ Alta/ Esencial ☐ Media/ Deseado ☐ Baja/ Opcional
Descripción	Tras el registro inicial de usuario la aplicación llevará al menú de perfil de actividad donde el usuario rellenará un formulario

	indicando datos tales como horario más activo (mañana o tarde).
--	---

Número de requisito	RF01.5
Nombre de requisito	Modificación de perfil de actividad
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	Cliente
Prioridad del requisito	☐ Alta/ Esencial ☒ Media/ Deseado ☐ Baja/ Opcional
Descripción	El sistema permitirá al usuario actualizar en cualquier momento su perfil de actividad.

Número de requisito	RF02.1
Nombre de requisito	Definición de horarios fijos
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	Cliente
Prioridad del requisito	☒ Alta/ Esencial ☐ Media/ Deseado ☐ Baja/ Opcional
Descripción	El sistema debe permitir registrar los bloques de tiempo inamovibles (ej. Jornada laboral de 8:00 a 15:00, horas de sueño) durante el proceso de configuración inicial.

Número de requisito	RF02.2
Nombre de requisito	Inserción de tarea flexible
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	Cliente
Prioridad del requisito	☒ Alta/ Esencial ☐ Media/ Deseado ☐ Baja/ Opcional
Descripción	El sistema permitirá al usuario introducir una tarea indicando su nombre, duración estimada en minutos/horas y su frecuencia semanal (ej. "Gimnasio", "1,5 horas", "3 días").

Número de requisito	RF02.3
Nombre de requisito	Prevención de solapamientos de horarios
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	Cliente
Prioridad del requisito	☒ Alta/ Esencial ☐ Media/ Deseado ☐ Baja/ Opcional
Descripción	El algoritmo (Go) tiene estrictamente prohibido asignar una tarea flexible en un hueco de tiempo que pise o se superponga a un bloque fijo o flexible definido en el Onboarding o evento ya confirmado.

Número de requisito	RF03.1
Nombre de requisito	Traducción de cálculos a propuesta humana
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	Cliente
Prioridad del requisito	☒ Alta/ Esencial ☐ Media/ Deseado ☐ Baja/ Opcional
Descripción	El sistema enviará los datos matemáticos calculados por el motor de Go a la API de Gemini para que este formule una propuesta al usuario en lenguaje natural, justificando el porqué del horario elegido en base a su energía y disponibilidad.

Número de requisito	RF03.2
Nombre de requisito	Confirmación de propuesta de calendario
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	Cliente
Prioridad del requisito	☒ Alta/ Esencial ☐ Media/ Deseado ☐ Baja/ Opcional
Descripción	La interfaz del chatbot presentará siempre dos botones de acción claros ("Sí" y "No") debajo de cada propuesta de horario. Al pulsar "Sí", el sistema consolidará el evento, guardándolo como definitivo en la base de datos.

Número de requisito	RF03.3
Nombre de requisito	Notificación de imprevisto por lenguaje natural
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	Cliente
Prioridad del requisito	☒ Alta/ Esencial ☐ Media/ Deseado ☐ Baja/ Opcional
Descripción	El usuario podrá escribir comandos libres en el chat (ej. "Hoy no puedo ir al gimnasio") y el sistema deberá extraer la intención y la tarea afectada mediante procesamiento de lenguaje natural para iniciar un recálculo.

Número de requisito	RF03.4
Nombre de requisito	Recálculo dinámico de la agenda
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	Cliente
Prioridad del requisito	☒ Alta/ Esencial ☐ Media/ Deseado ☐ Baja/ Opcional
Descripción	Tras registrar un imprevisto, el motor algorítmico reorganizará automáticamente el resto de la semana, buscando nuevos

	huecos disponibles para la tarea desplazada y formulando una nueva propuesta al usuario.
--	--

Número de requisito	RF04.1
Nombre de requisito	Guardado de seguridad del estado previo (Snapshot)
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	Cliente
Prioridad del requisito	☒ Alta/ Esencial ☐ Media/ Deseado ☐ Baja/ Opcional
Descripción	Inmediatamente antes de aplicar en el calendario un cambio masivo aceptado por el usuario, el sistema guardará de forma invisible una copia exacta del estado completo del calendario de esa semana.

Número de requisito	RF04.2
Nombre de requisito	Ejecución de Rollback
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	Cliente
Prioridad del requisito	☒ Alta/ Esencial ☐ Media/ Deseado ☐ Baja/ Opcional
Descripción	La interfaz de usuario mostrará una opción para "Deshacer" tras cada cambio algorítmico. Al ejecutarla, el sistema descartará la última actualización y restaurará el calendario a la copia de seguridad guardada especificada en el requisito RF04.1.

3.1 Requisitos comunes de los interfaces

3.1.1 Interfaces de usuario

La interfaz de usuario de Alyama se presentará como una aplicación web de página única (SPA) desarrollada en React. Los elementos principales son:

- Vista de Calendario Semanal: visualización del estado actual de la semana, con bloques fijos y tareas flexibles representados visualmente con distinción cromática.
- Panel de Chat: ventana de conversación lateral donde el chatbot (Gemini) presenta propuestas y el usuario puede responder mediante botones (Sí/No) o texto libre.
- Formularios de Onboarding: flujo de pasos guiados para recopilar el perfil cronobiológico y los horarios fijos en el primer acceso.
- Botón de Deshacer (Rollback): elemento visible en la interfaz cuando se elimina una tarea que permite revertir el último cambio algorítmico aplicado.

3.1.2 Interfaces de hardware

Al tratarse de una aplicación web, no existen requisitos estrictos de hardware para el cliente final más allá de un dispositivo (ordenador, tablet o smartphone) con conexión a Internet y capacidad para ejecutar un navegador web. En cuanto a la infraestructura del servidor, el despliegue del sistema se realiza en la plataforma en la nube **Render**. El backend (Go) contenerizado en Docker y el frontend operan sobre instancias virtuales proporcionadas por esta plataforma, requiriendo al menos 1GB de memoria RAM y 1 vCPU para garantizar un procesamiento fluido del algoritmo de asignación y la gestión de peticiones concurrentes.

3.1.3 Interfaces de software

El sistema se ejecuta en el navegador del cliente (SPA en React), por lo que es compatible con cualquier sistema operativo de escritorio (Windows 10/11, macOS, Linux) que disponga de un navegador web moderno (Chrome, Firefox, Edge, Safari). No se requiere la instalación de software adicional ni dependencias en el equipo del usuario final.

3.1.4 Interfaces de comunicación

Cliente-Servidor: La comunicación entre el frontend (React) y el backend (Go) se realizará a través del protocolo HTTP/HTTPS utilizando una arquitectura de API REST. El intercambio de información se hará estrictamente en formato JSON.

Servidor-Base de Datos: El backend se comunicará con el servidor remoto de base de datos (Supabase / PostgreSQL) utilizando el protocolo estándar y seguro de conexión de PostgreSQL.

Servidor-IA: Las consultas al modelo de inteligencia artificial se realizan desde el backend hacia la API externa de Gemini mediante peticiones HTTPS seguras y autenticadas con API Key.

3.2 Requisitos funcionales

3.2.1 Requisito funcional 1

Este grupo de requisitos (RF01.1 a RF01.5) engloba todas las funcionalidades relacionadas con la cuenta del usuario. Incluye el sistema de registro y autenticación segura, así como el flujo de "Onboarding" inicial donde el sistema recoge información vital (horarios fijos y perfil de actividad/cronotipo) para personalizar el motor de la inteligencia artificial.

3.2.2 Requisito funcional 2

Las funcionalidades de este bloque (RF02.1 a RF02.3) definen el núcleo de la agenda. Detallan cómo el sistema permite la inserción de rutinas inamovibles (bloques fijos) y tareas flexibles. Además, establecen la regla de negocio más importante del motor de Go: la prevención estricta de solapamientos entre eventos ya confirmados y nuevas propuestas.

3.2.3 Requisito funcional 3

Este apartado (RF03.1 a RF03.4) especifica la interacción conversacional del sistema. Detalla cómo el backend procesa los cálculos de disponibilidad y los envía al LLM (Gemini) para formular propuestas en lenguaje natural. También cubre la capacidad del chatbot para recibir imprevistos de los usuarios y recalcular dinámicamente el resto de la semana.

3.2.4 Requisito funcional 4

Los requisitos de este grupo (RF04.1 a RF04.3) garantizan la confianza del usuario en el sistema automatizado. Establecen la obligación de generar "snapshots" (copias de seguridad invisibles) antes de cada cambio masivo, permitiendo al usuario ejecutar un comando de "Deshacer" para revertir el estado de la agenda y solicitar una nueva distribución si la propuesta de la IA no le convence.

3.3 Requisitos no funcionales

3.3.1 Requisitos de rendimiento

El sistema debe ser capaz de procesar las peticiones de lenguaje natural, calcular los huecos libres y devolver una propuesta de calendario en un tiempo máximo de **30 segundos**, estando este tiempo condicionado principalmente por la latencia de la API de Gemini.

La carga inicial de la interfaz web (SPA en React) no debe superar los 3 segundos en conexiones estándar.

El motor de asignación (Go) debe calcular los huecos disponibles en la semana actual en menos de 1 segundo gracias al uso de estructuras de datos optimizadas en memoria.

3.3.2 Seguridad

Gracias al sistema de gestión de cuentas cada usuario tendrá una o más cuentas asociadas a un par de claves nombre de usuario-contraseña que impedirá que cualquier otro usuario que no sepa la contraseña entre en una cuenta diferente a la suya. Estas contraseñas serán cifradas antes de meterse en la base de datos para impedir que sean sacadas mediante inyecciones de código o fuerza bruta.

3.3.3 Fiabilidad

El sistema debe garantizar la integridad del calendario del usuario en todo momento. En caso de fallo en el algoritmo de asignación o interrupción de la conexión con la IA, el sistema no aplicará cambios parciales. La fiabilidad ante cambios no deseados recae en la función de **Rollback**, garantizando que el usuario siempre pueda recuperar el 100% del estado previo de su agenda si rechaza una reprogramación masiva.

3.3.4 Disponibilidad

Al ser una aplicación web desplegada en la nube y apoyada en servicios SaaS/PaaS (**Render** para el alojamiento del frontend y backend, **Supabase** para la base de datos y **Google** para la IA), la disponibilidad del servicio dependerá directamente del *uptime* garantizado por estos proveedores (habitualmente superior al 99.9%). El sistema debe estar preparado para manejar caídas de servicios de terceros (ej. sobrecarga en la API de Gemini) mostrando mensajes de error controlados en la interfaz de usuario (ej. "La IA está saturada, inténtalo de nuevo") sin que la aplicación se congele o se cierre de forma abrupta.

3.3.5 Mantenibilidad

El backend está desarrollado en Go con una arquitectura modular, separando claramente los controladores HTTP (**handlers**), el acceso a datos (**repository**) y la lógica de negocio/IA (**engine**). Esto facilita que cualquier desarrollador pueda localizar y corregir errores rápidamente. El frontend (React) utiliza componentes reutilizables (ej. tarjetas de calendario, burbujas de chat) para agilizar la adición de nuevas funcionalidades visuales.

3.3.6 Portabilidad

El frontend es universalmente portable a nivel de usuario, ya que funciona en cualquier navegador web moderno (Chrome, Safari, Edge, Firefox) sin importar el sistema operativo subyacente. El backend está contenerizado mediante Docker y orquestado con docker-compose. Esto garantiza que el entorno de ejecución sea idéntico independientemente de si se despliega en local para desarrollo o en la nube.

para producción. Gracias a esta alta portabilidad, el despliegue en la plataforma **Render** se realiza de forma directa y estandarizada a través de la imagen del contenedor.

3.4 Otros requisitos

Requisitos de Integración y Despliegue Continuo (CI/CD) El ciclo de vida del software exige un flujo de Integración y Despliegue Continuo para asegurar la estabilidad del sistema en producción:

Pipeline (Integración Continua): El repositorio debe contar con flujos automatizados mediante **GitHub Actions**. Cada actualización de código (*push* o *Pull Request*) disparará la compilación del código en Go y la ejecución de pruebas para asegurar la integridad del sistema.

Despliegue Continuo: La plataforma en la nube (**Render**) debe estar conectada al repositorio o registro de contenedores para automatizar la puesta en producción. Cada vez que se integre una nueva versión estable, Render desplegará automáticamente la nueva imagen Docker del backend y la actualización del frontend.

4 Apéndices

Apéndice A - Documentación de Requisitos: Este documento de Especificación de Requisitos de Software (SRS), entregado de forma oficial a través de la plataforma virtual Ágora.

Apéndice B - Repositorio de Código Fuente: El código completo y versionado de la aplicación Alyama. Incluye la arquitectura monorepo con el backend (Go), el frontend (React), la configuración de contenedores (Docker) y los pipelines de automatización (**GitHub Actions**). Todo el desarrollo se encuentra alojado y accesible en el repositorio oficial del equipo en GitHub.